SKYROUTE DBMS MINI PROJECT

SkyRoute: Airline Ticket and Baggage Management System

DBMS Mini Project - Database Management System (CS/AI/AIM/DS202)

Database: skyroute_db | Tables: 16 | MySQL 8.0

Seed volume: 140 departures, ~14,000 bookings, ~17,000 baggage records (deterministic generator, seed 42).

Abstract

Managing flight bookings, seat allocation, gate assignment and baggage handling across several airports by hand causes duplicate seat bookings, gate clashes, wrong baggage-fee computation and inconsistent data between stations. SkyRoute is a centralized relational database for five Indian airports and three airlines that automates scheduling, reservations, baggage tracking, payments and check-in. The schema separates timetable templates (Flights) from dated departures (Departures) so the system can hold an unlimited number of dated flights; models Terminal/Gate infrastructure per airport; and organises staff with a supertype/subtype hierarchy. Integrity rules live inside the database: a unique seat-per-departure key blocks double booking, a trigger blocks overselling past aircraft capacity, composite gate keys block gate clashes, and check constraints block impossible routes and weights. A stored procedure books a seat and opens its payment atomically, two reporting views serve the departure board and the booking receipt (including the overweight-baggage fee), transaction demos prove atomicity and isolation, and targeted indexes with before/after EXPLAIN checks prove the system stays fast at volume.

1. Title Selection

SkyRoute: Airline Ticket and Baggage Management System. The title reflects the real-world problem solved - ticketing and baggage handling for scheduled airline operations across multiple airports - rather than a single-route shuttle.

2. Problem Statement

Manual and single-station booking workflows suffer five concrete failures:

- Duplicate seat booking on the same flight when two agents sell concurrently - closed by UNIQUE(departure_id, seat_number).
- Gate assignment clashes written down wrongly between flights - closed by UNIQUE(terminal_id, gate_number) plus gate rotation in scheduling.
- Baggage fees miscalculated above the 20 kg free allowance - closed by the fee rule encoded once in booking_receipt_view (Rs 500 per excess kg).
- No single database shared by flights, passengers and bookings across stations
- closed by one skyroute_db serving all five airports.
- No automated guard when a flight sells more seats than the aircraft holds - closed by the trg_booking_capacity trigger against aircraft capacity.

3. Objectives

- Automate flight scheduling and gate allocation across five airports.
- Make double seat booking and overselling impossible at the storage-engine level.
- Compute baggage fees deterministically from weighed kilograms.
- Retrieve flight boards, manifests, revenue and utilisation with single queries.
- Guarantee integrity with least-privilege access control (admin / staff / viewer).
- Prove performance at volume with indexing and query-cost analysis.

4. Requirements Gathering

Functional: add, edit and view airports, airlines, aircraft, flights and dated departures; assign aircraft, gate and crew to each dated departure; reserve seats and create bookings; log baggage weight per booking; record payments and check-ins; manage staff and roles.

Non-functional: fast query response at tens of thousands of rows; data integrity under concurrent sellers; secure access.

5. Entity-Relationship Modeling

Entities: Airport, Airline, Aircraft, Flight, Departure, Terminal, Gate, Passenger, Booking, Baggage (weak), Payment, Check_In, Employee (supertype) with Pilot, Crew_Member, Ground_Staff subtypes.

Relationships: Airline 1-N Aircraft; Flight N-1 Airports (departure and arrival roles); Flight 1-N Departures; Airport 1-N Terminals 1-N Gates; Passenger

1-N Bookings N-1 Departures; Booking 1-N Baggage (identifying) and 1-1 Payment / Check_In. (Diagram: ER_Diagram.png.)

6. Schema Design (Relational Model)

Table	Primary key	Rule that matters
Airports	airport_id	UNIQUE iata, name
Airlines	airline_id	UNIQUE name, iata
Aircraft	aircraft_id	UNIQUE tail; CHECK cap 1-850
Flights	flight_id	UNIQUE number; dep <> arr airports
Departures	departure_id	UNIQUE(flight, dep datetime)
Terminals / Gates	own ids	UNIQUE per parent (anti-clash)
Passengers	passenger_id	UNIQUE passport, email
Bookings	booking_id	UNIQUE(departure, seat)
Baggage	(booking, seq)	weak entity; 0 < kg <= 100
Payments / Check-Ins	own ids	UNIQUE booking (one-to-one)
Employees + subtypes	employee_id	subtype PK = supertype PK

16 InnoDB tables. Surrogate integer primary keys keep foreign keys narrow at scale; natural business keys (IATA codes, tail numbers, licences, emails, transaction references) carry UNIQUE constraints instead.

- 1NF: repeating groups removed into masters (airline/aircraft data live once, not inside every flight row).
- 2NF: every non-key attribute depends on the whole key (gate status depends on gate_id, never on a booking).
- 3NF: no transitive dependencies (city depends on airport; flights reference the airport rather than copying its city).
- BCNF-clean: every determinant in every table is a key.

Key tables and rules: Airports (UNIQUE iata, name); Airlines (UNIQUE name, iata); Aircraft (UNIQUE tail; CHECK capacity 1-850); Flights (UNIQUE number; two airport FKs; CHECK departure <> arrival); Departures (UNIQUE(flight, scheduled_departure); CHECK arrival > departure); Terminals, Gates (UNIQUE per parent - the anti-clash keys); Passengers (UNIQUE passport, email); Bookings (UNIQUE(departure, seat)); Baggage (PK is (booking_id, seq); CHECK 0 < kg <= 100); Payments and Check_Ins (UNIQUE booking_id - one-to-one); Employees + 3 subtypes (subtype PK = supertype PK);

7. SQL Implementation

DDL: 01-travel-core.sql creates the 12 travel tables (CASCADE where children cannot outlive parents, RESTRICT on airports - deleting an airport must never

wipe flights). 02-staff-roles.sql adds the staff hierarchy and the DCL: three users (sky_admin full, sky_staff read-write, sky_viewer read-only) with SHOW GRANTS proof on the exact database name.

DML: generate.py (seed 42) writes seed_data.sql deterministically - 5 airports, 3 airlines, 10 aircraft, 10 scheduled flights, 140 dated departures over 14 days, 500 passengers, ~14,000 bookings at 35-80% load factors with seats sampled without replacement, ~14,000 payments, ~17,000 bags (80% under 20 kg, 20% overweight tail for the fee demo), ~5,000 check-ins on past departures, 8 staff with subtype rows and crew assignments. Machine-verified: zero duplicate (departure, seat) pairs, all foreign keys in range.

DCL: least-privilege roles as above; viewer INSERT is denied.

8. Advanced SQL Features

Views: flight_board_view joins timetable, airports, airline, aircraft, gate and terminal into the single departure-board query. booking_receipt_view returns price plus the computed baggage fee via a scalar subquery (valid under ONLY_FULL_GROUP_BY).

Stored procedure: MakeBooking(passenger, departure, seat, class, price, OUT id) inserts the booking and opens its Pending payment inside one transaction with an EXIT HANDLER that rolls back and re-signals on any error.

Trigger: trg_booking_capacity (BEFORE INSERT ON Bookings) compares seats sold against the assigned aircraft's capacity and SIGNALs SQLSTATE 45000 when full. It fires for every insert path because the rule lives in the database.

Joins and subqueries: 10 join queries across every module; analytics pack with GROUP BY, HAVING, CASE aggregates and a subquery.

9. Transaction Management

InnoDB provides ACID. Demo A commits a booking plus its payment together. Demo B rolls back on payment failure and the seat stays free (atomicity). Demo C uses two simultaneous mysql sessions to show REPEATABLE READ isolation. Script: advanced/04-transactions.sql.

10. Indexing and Optimization

InnoDB auto-indexes single-column foreign keys, so those were deliberately not re-indexed. Four workload-driven indexes: booking_status, scheduled_departure, (last_name, first_name), weight_kg - each with a before/after EXPLAIN pair in the test log showing fewer rows examined. Script: advanced/05-indexes.sql.

11. Testing and Validation

TESTING.md records 30 checks in 8 groups: joins, analytics, constraint rejection (duplicate seat ERROR 1062, impossible route, bad weight, RESTRICT on airports, CASCADE orphan check), procedure atomicity, transaction demos, EXPLAIN pairs, role grants, and scale counts. Every check states its expected result.

12. Documentation

This report; README.md (run order, users, rules, layout, screenshot workflow); TESTING.md (validation protocol); decision journal C1-C13 embedded as comments in the SQL sources; ER diagram; slide deck; Flask brief report.

13. Presentation and Reporting

This report, the 11-slide deck, all source SQL and the generator, README, test log, and a 2-3 minute demo video (departure board, live booking, duplicate-seat refusal, fee receipt).

Conclusion

Every problem-statement item is closed by a named database mechanism, every mechanism has a test, and the system holds production-shaped volume (~14k bookings) while answering analytical questions in single queries. The Flask web project reuses this design on SQLite with raw SQL, keeping the same integrity rules visible in application code.

References

- Ramakrishna R., Gehrke J. Database Management Systems, 3rd Ed, McGraw-Hill.
- Silberschatz, Korth, Sudarshan. Database System Concepts, 7th Ed.
- MySQL 8.0 Reference Manual: constraints, triggers, stored programs, EXPLAIN.